

Project Report : $\mathbb{P}_2$ -Lagrange finite elements

Maxime Houlette

Contents

1	Short introduction to $\mathbb{P}_2$-Lagrange finite elements	2
1.1	$\mathbb{P}_k$ -Lagrange finite elements	2
1.2	Particular case $d = 2$ and $k = 2$	2
1.3	Shape functions	3
2	Local mass and stiffness matrices	3
2.1	Local mass matrix	4
2.2	Local stiffness matrix	5
3	Global mass and stiffness matrices	5
3.1	Sparsity pattern	6
3.1.1	Main idea	6
3.1.2	Some technical tools used to build the sparsity pattern	6
3.2	Global mass and stiffness matrices	6
4	Transport term in the vorticity formulation of Navier-Stokes	6
5	Validation of the code	8
5.1	Poisson solver	8
5.2	Navier-Stokes solver	10
5.2.1	Viscous flow	10
5.2.2	Transport flow	11

1 Short introduction to $\mathbb{P}_2$ -Lagrange finite elements

The goal of this section is to provide a short reminder of what $\mathbb{P}_2$ -Lagrange finite elements are, and what the main differences are with $\mathbb{P}_1$ -Lagrange finite elements.

1.1 $\mathbb{P}_k$ -Lagrange finite elements

Given a conforming mesh $\mathcal{T}_h$, we recall that for all $K \in \mathcal{T}_h$, the $\mathbb{P}_k$ -Lagrange finite element is the simplicial finite element defined as follows:

$$(K, \mathcal{P}_K, \Sigma_K)$$

such that :

- K is a d -simplex,
- $\mathcal{P}_K$ is the space $\mathbb{P}_k(K)$ of polynomials of total degree at most k , which has dimension $\binom{d+k}{k}$ (or equivalently $\binom{d+k}{d}$),
- Σ_K is the set of linear forms associated with the evaluation of an element of $\mathcal{P}_K$ at the nodes of the principal lattice of order k .

We introduce the principal lattice of order k associated to a simplex K

$$T_k(K) = \left\{ \mathbf{x} \in \mathbb{R}^d : \lambda_j(\mathbf{x}) \in \left\{ 0, \frac{1}{k}, \dots, \frac{k-1}{k}, 1 \right\}, \forall j \in \llbracket 1, d+1 \rrbracket \right\} = (\mathbf{x}_i)_{1 \leq i \leq \binom{d+k}{d}}$$

where the $\lambda_j(\mathbf{x})$ are the barycentric coordinates of $\mathbf{x}$ in the simplex K . Then

$$\Sigma_K = (\sigma_i)_{1 \leq i \leq \binom{d+k}{d}}$$

where

$$\forall p \in \mathcal{P}_K, \sigma_i(p) = p(\mathbf{x}_i)$$

The unsolvence property is satisfied, so this is a well-posed finite element. Denoting Φ the isomorphism

$$\begin{aligned} \Phi : \mathcal{P}_K &\rightarrow \mathbb{R}^{\binom{d+k}{d}} \\ p &\mapsto \left(p(\mathbf{x}_1), \dots, p(\mathbf{x}_{\binom{d+k}{d}}) \right) \end{aligned}$$

and considering the canonical basis of $\mathbb{R}^{\binom{d+k}{d}}$, we build a basis of $\mathcal{P}_K$ by considering its unique preimage by Φ , that we denote $(\varphi_i)_{1 \leq i \leq \binom{d+k}{d}}$. Hence, this basis of $\mathcal{P}_K$ satisfies

$$\forall (i, j) \in \llbracket 1, \binom{d+k}{d} \rrbracket^2, \varphi_i(\mathbf{x}_j) = \delta_{ij}$$

1.2 Particular case $d = 2$ and $k = 2$

In the case where $d = 2$ and $k = 2$, for all $K \in \mathcal{T}_h$, we are considering $\mathcal{P}_K = \mathbb{P}_2(K)$, which has the dimension 6. In that case, the principle lattice of order 2 of a given K is

$$T_2(K) = \left\{ \mathbf{x} \in \mathbb{R}^d : \lambda_j(\mathbf{x}) \in \left\{ 0, \frac{1}{2}, 1 \right\}, \forall j \in \llbracket 1, 3 \rrbracket \right\} = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_{12}, \mathbf{x}_{23}, \mathbf{x}_{31}\}$$

The points $(\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)$, which are the mesh vertices, and $(\mathbf{x}_{12}, \mathbf{x}_{23}, \mathbf{x}_{31})$, the edge midpoints, can be represented as follows:

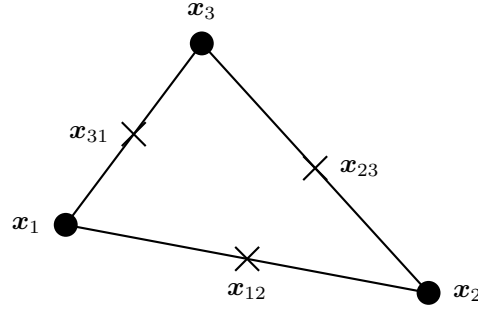


Figure 1: Local simplex K

We are now interested in computing the φ_i . It is clear that the main challenge of this framework will be handling the fact that the vertices are no longer the only degrees of freedom, as mid-edge points have been added.

1.3 Shape functions

We compute the $\mathbb{P}_2$ shape functions associated with the triangle K shown in Figure 1. We provide the computational details for the shape function associated with $\mathbf{x}_1$ in barycentric coordinates, denoted as $\tilde{\varphi}_{\mathbf{x}_1}$. That is to say that, considering the reference triangle $\hat{K} := ((0, 0), (1, 0), (0, 1))$, and knowing that every $\mathbf{x} \in K$ can be uniquely represented with $(\lambda_1(\mathbf{x}), \lambda_2(\mathbf{x})) \in \hat{K}$ (as $\lambda_3(\mathbf{x}) = 1 - \lambda_1(\mathbf{x}) - \lambda_2(\mathbf{x})$), we can write

$$\forall \mathbf{x} \in K, \varphi_{\mathbf{x}_1}(\mathbf{x}) = \tilde{\varphi}_{\mathbf{x}_1}(\lambda_1(\mathbf{x}), \lambda_2(\mathbf{x}))$$

By definition, $\tilde{\varphi}_{\mathbf{x}_1} \in \mathbb{P}_2(\hat{K})$ is uniquely determined by the following six constraints:

$$1 = \underbrace{\tilde{\varphi}_{\mathbf{x}_1}(1, 0)}_{\varphi_{\mathbf{x}_1}(\mathbf{x}_1)}$$

and

$$0 = \underbrace{\tilde{\varphi}_{\mathbf{x}_1}(0, 1)}_{\varphi_{\mathbf{x}_1}(\mathbf{x}_2)} = \underbrace{\tilde{\varphi}_{\mathbf{x}_1}(0, 0)}_{\varphi_{\mathbf{x}_1}(\mathbf{x}_3)} = \underbrace{\tilde{\varphi}_{\mathbf{x}_1}\left(\frac{1}{2}, \frac{1}{2}\right)}_{\varphi_{\mathbf{x}_1}(\mathbf{x}_{12})} = \underbrace{\tilde{\varphi}_{\mathbf{x}_1}\left(0, \frac{1}{2}\right)}_{\varphi_{\mathbf{x}_1}(\mathbf{x}_{23})} = \underbrace{\tilde{\varphi}_{\mathbf{x}_1}\left(\frac{1}{2}, 0\right)}_{\varphi_{\mathbf{x}_1}(\mathbf{x}_{31})}$$

Solving a linear system of six equations yields

$$\tilde{\varphi}_{\mathbf{x}_1}(\lambda_1, \lambda_2) = \lambda_1(2\lambda_1 - 1)$$

We end up with the table :

	$\mathbf{z} = \mathbf{x}_i$	$\mathbf{z} = \mathbf{x}_{i\bar{i}}$
$\tilde{\varphi}_{\mathbf{z}}(\lambda_1, \lambda_2)$	$\lambda_i(2\lambda_i - 1)$	$4\lambda_i\lambda_{i+1}$

Table 1: Reference $\mathbb{P}_2$ shape functions

with the notation $\bar{i} = i + 1$ and the conventions $\lambda_4 = \lambda_1$ and $\mathbf{x}_{34} = \mathbf{x}_{31}$.

2 Local mass and stiffness matrices

Given a triangle $K = (\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3)$, we will extensively use the fact that

$$\mathbf{x} \in K \Leftrightarrow \lambda_j(\mathbf{x}) \geq 0 \text{ and } \sum_{i=1}^3 \lambda_j(\mathbf{x}) = 1$$

For a function f defined on a triangle K , the integral over K can be computed using the change of variables $\mathbf{x} \in K \Leftrightarrow (\lambda_1(\mathbf{x}), \lambda_2(\mathbf{x})) \in \hat{K}$, which yields:

$$\int_{\mathbf{x} \in K} f(\mathbf{x}) d\mathbf{x} = \Lambda \int_{(\lambda_1, \lambda_2) \in \hat{K}} \tilde{f}(\lambda_1, \lambda_2) d\lambda_1 d\lambda_2 = \Lambda \int_{\lambda_1=0}^1 \int_{\lambda_2=0}^{1-\lambda_1} \tilde{f}(\lambda_1, \lambda_2) d\lambda_2 d\lambda_1 \quad (1)$$

where, for $\mathbf{x} \in K$ and the corresponding $(\lambda_1(\mathbf{x}), \lambda_2(\mathbf{x})) \in \hat{K}$, we have $f(\mathbf{x}) = \tilde{f}(\lambda_1(\mathbf{x}), \lambda_2(\mathbf{x}))$ (as introduced before), and $\Lambda = \|\mathbf{n}\|$, with $\mathbf{n} = (\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_3 - \mathbf{x}_1)$, is related to the actual geometric shape of K . This last statement follows from the fact that any $\mathbf{x} \in K$ can be expressed as:

$$\mathbf{x} = \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2 + \lambda_3 \mathbf{x}_3 = \lambda_1(\mathbf{x}_1 - \mathbf{x}_3) + \lambda_2(\mathbf{x}_2 - \mathbf{x}_3)$$

Thus, the Jacobian of the change of variables is

$$J = (\mathbf{x}_1 - \mathbf{x}_3 \quad \mathbf{x}_2 - \mathbf{x}_3) = \begin{pmatrix} x_1 - x_3 & x_2 - x_3 \\ y_1 - y_3 & y_2 - y_3 \\ z_1 - z_3 & z_2 - z_3 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

which leads to the term

$$\sqrt{\det(J^T J)} = \|(\mathbf{x}_1 - \mathbf{x}_3) \times (\mathbf{x}_2 - \mathbf{x}_3)\| = \|(\mathbf{x}_2 - \mathbf{x}_1) \times (\mathbf{x}_3 - \mathbf{x}_1)\| = \|\mathbf{n}\| = \Lambda$$

in the change of variables.

2.1 Local mass matrix

Once the formulas for the shape functions are obtained, and using the change of variables (1), the coefficients of the local mass matrix $M \in \mathbb{R}^{6 \times 6}$ can be readily computed as follows:

$$M_{ij} = \int_K \varphi_i \varphi_j = \Lambda \int_{\hat{K}} \tilde{\varphi}_i \tilde{\varphi}_j$$

where we use the correspondence $1 \leftrightarrow \mathbf{x}_1$, $2 \leftrightarrow \mathbf{x}_2$, $3 \leftrightarrow \mathbf{x}_3$, $4 \leftrightarrow \mathbf{x}_{12}$, $5 \leftrightarrow \mathbf{x}_{23}$, and $6 \leftrightarrow \mathbf{x}_{31}$. Computing the seven distinct coefficients — starting from 36, reduced to 21 by symmetry, and finally to 7 by change of orientation — is straightforward but technical. We directly provide the results for one representative interaction of each coefficient (for instance, the interactions $\mathbf{x}_1 \leftrightarrow \mathbf{x}_1$ and $\mathbf{x}_2 \leftrightarrow \mathbf{x}_2$ are identical, as are $\mathbf{x}_1 \leftrightarrow \mathbf{x}_2$ and $\mathbf{x}_1 \leftrightarrow \mathbf{x}_3$). We obtain:

	$\mathbf{x}_1 \leftrightarrow \mathbf{x}_1$	$\mathbf{x}_1 \leftrightarrow \mathbf{x}_2$	$\mathbf{x}_{12} \leftrightarrow \mathbf{x}_{12}$	$\mathbf{x}_{12} \leftrightarrow \mathbf{x}_{23}$	$\mathbf{x}_1 \leftrightarrow \mathbf{x}_{12}$	$\mathbf{x}_1 \leftrightarrow \mathbf{x}_{23}$
$\frac{360}{\Lambda} M_{ij}$	6	-1	32	16	0	-4

Table 2: Coefficients of the mass matrix M

This finally yields the local mass matrix

$$M = \frac{\Lambda}{360} \begin{pmatrix} M_{vv} & M_{ev}^T \\ M_{ev} & M_{ee} \end{pmatrix}$$

where

$$M_{vv} = \begin{pmatrix} 6 & -1 & -1 \\ -1 & 6 & -1 \\ -1 & -1 & 6 \end{pmatrix}$$

$$M_{ev} = \begin{pmatrix} 0 & 0 & -4 \\ -4 & 0 & 0 \\ 0 & -4 & 0 \end{pmatrix}$$

and

$$M_{ee} = \begin{pmatrix} 32 & 16 & 16 \\ 16 & 32 & 16 \\ 16 & 16 & 32 \end{pmatrix}$$

2.2 Local stiffness matrix

We now compute the stiffness matrix $S \in \mathbb{R}^{6 \times 6}$, that is defined by

$$S_{ij} = \int_K \nabla \varphi_i \cdot \nabla \varphi_j$$

A direct change of variables cannot be performed here due to the presence of gradients (while the Jacobian-based approach will be discussed later, an alternative method is employed here). Instead, we utilize the following useful expressions, valid for all $1 \leq i \leq 3$:

$$\nabla \lambda_i(\mathbf{x}) = \frac{(\mathbf{x}_{i+1} - \mathbf{x}_{i-1}) \times \mathbf{n}}{\Lambda^2}$$

with the conventions $\mathbf{x}_0 = \mathbf{x}_3$ and $\mathbf{x}_4 = \mathbf{x}_1$. Using this formula, we can express the constant gradient of each barycentric coordinate as follows:

$$\nabla \varphi_{\mathbf{x}_i} = (4\lambda_i - 1) \nabla \lambda_i$$

and

$$\nabla \varphi_{\mathbf{x}_{ii}} = 4(\lambda_i \nabla \lambda_{i+1} + \lambda_{i+1} \nabla \lambda_i)$$

Let us denote $L_{ij} = \nabla \lambda_i \cdot \nabla \lambda_j$. After some computations, we obtain the coefficients of the local stiffness matrix $S \in \mathbb{R}^{6 \times 6}$ as follows:

$$S = \frac{\Lambda}{6} \begin{pmatrix} S_{vv} & S_{ev}^T \\ S_{ev} & S_{ee} \end{pmatrix}$$

where

$$S_{vv} = \begin{pmatrix} 3L_{11} & -L_{12} & -L_{13} \\ -L_{12} & 3L_{22} & -L_{23} \\ -L_{13} & -L_{23} & 3L_{33} \end{pmatrix}$$

$$S_{ev} = \begin{pmatrix} 4L_{12} & 4L_{13} & 0 \\ 0 & 4L_{23} & 4L_{23} \\ 4L_{13} & 0 & 4L_{13} \end{pmatrix}$$

and

$$S_{ee} = \begin{pmatrix} 8(L_{11} + L_{22} + L_{12}) & 4(L_{23} + L_{22} + 2L_{13} + L_{12}) & 4(L_{12} + 2L_{23} + L_{11} + L_{13}) \\ 4(L_{23} + L_{22} + 2L_{13} + L_{12}) & 8(L_{22} + L_{33} + L_{23}) & 4(L_{13} + L_{33} + 2L_{12} + L_{23}) \\ 4(L_{12} + 2L_{23} + L_{11} + L_{13}) & 4(L_{13} + L_{33} + 2L_{12} + L_{23}) & 8(L_{33} + L_{11} + L_{13}) \end{pmatrix}$$

This matrix can be simplified by using the property $0 = \nabla 1 = \nabla \lambda_1 + \nabla \lambda_2 + \nabla \lambda_3$. We thus obtain:

$$S_{ee} = \begin{pmatrix} 8(L_{11} - L_{23}) & 8L_{13} & 8L_{23} \\ 8L_{13} & 8(L_{22} - L_{13}) & 8L_{12} \\ 8L_{23} & 8L_{12} & 8(L_{33} - L_{12}) \end{pmatrix}$$

3 Global mass and stiffness matrices

For the global matrices, a natural choice for ordering the degrees of freedom is to put the actual vertices of the mesh first, and then the mid-edge points. With this ordering, the first block of both the mass and stiffness matrices corresponds precisely to the $\mathbb{P}_1$ counterpart matrix.

In what follows, we will identify the degrees of freedom corresponding to the mid-edge points with the edges themselves.

3.1 Sparsity pattern

3.1.1 Main idea

The main difficulty of $\mathbb{P}_2$ -Lagrange finite elements lies in the construction of the sparsity pattern. In fact, instead of just handling interactions between vertices, we now have to handle these, in addition to interactions between edges and vertices (ev), and interactions between edges (ee). A first idea is to use the fact that the global matrices are symmetric and divided into three blocks: one block vv , one block ev , and one block ee . It seems natural to try to build the sparsity pattern by block.

However, we hit an algorithmic issue when trying to do that. Indeed, the `row_start` and `col` objects, which entirely encode the sparsity pattern of the global matrices, do not efficiently allow this kind of usage. Instead, by definition, they are designed to work row by row, filling the column object corresponding to the current row, essentially by incrementing the `P.nnz` instance. Therefore, a better idea is to follow this definition and to build the sparsity pattern in two steps, corresponding to the two categories of degrees of freedom: the vertices and the edges. Knowing that the matrices are symmetric, we can, for example, consider the lower triangular part and write one function that first fills the vv interactions (vertex rows), and another that handles the ev and ee (edge rows) interactions.

3.1.2 Some technical tools used to build the sparsity pattern

In this report, we won't get too much into the details of the code. We just introduce, in this subsection, the tools that were used to implement the algorithm.

We first have to be able to represent what an edge is, which can be done by introducing an **Edge** structure that encodes the indices of two neighbors v_0 and v_1 , satisfying $v_0 < v_1$. With this canonical definition, we ensure that for two neighbors v_0 and v_1 , the edges (v_0, v_1) and (v_1, v_0) are the same. Then, as for the $\mathbb{P}_1$ sparsity pattern, we need to have access to all the triangles sharing a given edge. This is done by building a very similar structure to **VTAdjacency**, which we call **ETAdjacency** (Edge to Triangle Adjacency). With these two tools, we are now able to handle both vertices and edges.

The last tools we introduce are the structures that allow us to track the interactions that have already been seen during the algorithm. These three structures are **VertexPair** (for vv interactions), **VertexEdgePair** (for ev interactions), and **EdgePair** (for ee interactions), as well as their associated hasher structures, which are used to build the **HashTable** objects.

All these tools allow us to build the sparsity pattern in a very efficient way, by using the full power of hash table lookups. Moreover, this construction also shows that it is not much more difficult to build higher-order $\mathbb{P}_k$ FEM. In fact, this would only require building some other pair structures to take all the interactions into account.

3.2 Global mass and stiffness matrices

Once the sparsity pattern is built, it is very easy to fill the global mass and stiffness matrices. We just have to loop over all the triangles and use the `operator()` method, which itself utilizes the `row_start` and `col` objects.

4 Transport term in the vorticity formulation of Navier-Stokes

We recall that the transport term in the Navier-Stokes formulation is given for all $j \in I$ by

$$T_j = \sum_{i, k \in I} \Omega_i \Psi_k \int_{\Omega} \varphi_i \nabla^{\perp} \varphi_k \cdot \nabla \varphi_j$$

where I is the full set of DOF indices. The first simplification that can be made is that the only terms that are a priori non-zero are those where at least one triangle is shared. We then have:

$$T_j = \sum_{K \in \mathcal{T}_j} \sum_{i, k \in I_K} \Omega_i \Psi_k \int_K \varphi_i \nabla^{\perp} \varphi_k \cdot \nabla \varphi_j$$

where $\mathcal{T}_j$ is the set of all triangles containing the DOF j , and I_K is the set of DOFs contained in K .

This term is easy to compute for $\mathbb{P}_1$ finite elements, as the only degrees of freedom are the vertices of the mesh and the gradients of the shape functions are constant. In the case of $\mathbb{P}_2$ finite elements, it is not as simple because the mid-edge points stand as new degrees of freedom and the gradients are linear, so they cannot be factored out of the integral. Consequently, there are many terms to compute explicitly. Still, since all terms are polynomials, they can be computed exactly. We choose to evaluate the integrals with an exact quadrature rule to avoid computing all terms by hand. We use the Dunavant rule, which consists of seven points $(\boldsymbol{\xi}_q)_q$ expressed in barycentric coordinates on the reference triangle, along with seven weights $(w_q)_q$. For a polynomial p of degree at most 5, we can write:

$$\int_K p = \Lambda \int_{\tilde{K}} \tilde{p} = \Lambda \sum_{q=1}^7 w_q p(\boldsymbol{\xi}_q)$$

Thus, we have that

$$T_j = \sum_{q=1}^7 \sum_{K \in \mathcal{T}_j} \sum_{i, k \in I_K} \Omega_i \Psi_k w_q \varphi_i(\boldsymbol{\xi}_q) \nabla^\perp \varphi_k(\boldsymbol{\xi}_q) \cdot \nabla \varphi_j(\boldsymbol{\xi}_q) \Lambda_K$$

The final expression we will use is

$$T_j = \sum_{K \in \mathcal{T}_j} \sum_{q=1}^7 w_q A_K(\boldsymbol{\xi}_q) [(\mathbf{n} \times \mathbf{B}_K(\boldsymbol{\xi}_q)) \cdot \nabla \varphi_j(\boldsymbol{\xi}_q)] \Lambda_K$$

where $A_K(\boldsymbol{\xi}_q) = \sum_{i \in I_K} \Omega_i \varphi_i(\boldsymbol{\xi}_q)$ and $\mathbf{B}_K(\boldsymbol{\xi}_q) = \sum_{k \in I_K} \Psi_k \nabla \varphi_k(\boldsymbol{\xi}_q)$, $\mathbf{n}$ is the unit normal to K , and $\Lambda_K = \sqrt{\det(J_K^T J_K)}$ for all $K \in \mathcal{T}_j$. In fact, we used the formula:

$$\nabla^\perp = \mathbf{n} \times \nabla$$

in a triangle K . We are now led to compute the values of the shape functions at the quadrature points, as well as their gradients.

It is very easy and efficient to compute the terms $\varphi_i(\boldsymbol{\xi}_q)$, as they only rely on the barycentric coordinates of the quadrature points in the triangle, which can be computed offline (see the function `precompute_phi_at_quad_points`). Computing the gradients is also straightforward, but we first need to perform some intermediary computations. For a given triangle K , we carry out the computation for any function f that we want to express in terms of:

$$\tilde{\nabla} f = \begin{pmatrix} \frac{\partial f}{\partial \lambda_1} \\ \frac{\partial f}{\partial \lambda_2} \end{pmatrix}$$

The chain rule allows to write:

$$\tilde{\nabla} f = \begin{pmatrix} \frac{\partial \mathbf{x}}{\partial \lambda_1} \cdot \nabla f \\ \frac{\partial \mathbf{x}}{\partial \lambda_2} \cdot \nabla f \end{pmatrix} = J^T \nabla f$$

where $J = \tilde{\nabla} \mathbf{x}$. In the triangle K , we have the following linear mapping:

$$\mathbf{x} = \lambda_1 \mathbf{x}_1 + \lambda_2 \mathbf{x}_2 + \lambda_3 \mathbf{x}_3 = \lambda_1(\mathbf{x}_1 - \mathbf{x}_3) + \lambda_2(\mathbf{x}_2 - \mathbf{x}_3)$$

Then, we have that:

$$J = \begin{pmatrix} \mathbf{x}_1 - \mathbf{x}_3 & \mathbf{x}_2 - \mathbf{x}_3 \end{pmatrix} = \begin{pmatrix} x_1 - x_3 & x_2 - x_3 \\ y_1 - y_3 & y_2 - y_3 \\ z_1 - z_3 & z_2 - z_3 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$$

which is the Jacobian of the triangle. Since ∇f lies in the plane of the triangle, we can write:

$$\nabla f = \alpha(\mathbf{x}_1 - \mathbf{x}_3) + \beta(\mathbf{x}_2 - \mathbf{x}_3) = J \mathbf{u}$$

where $\mathbf{u} = (\alpha, \beta)^T$, and finally, knowing that $J^T J$ is invertible, we find that:

$$\nabla f = J \mathbf{u} = J(J^T J)^{-1} \tilde{\nabla} f$$

This expression is very convenient because the vector $\tilde{\nabla} \tilde{f}$ can be computed in an offline procedure (see the function `precompute_grad_phi_at_quad_points`), so that we just have to handle the geometry of the current triangle with the Jacobian J . Moreover, the matrix $J^T J$ is very easy to invert because it is a 2×2 matrix. The expression for $J^T J$ is, noting $\mathbf{e}_1 = \mathbf{x}_1 - \mathbf{x}_3$ and $\mathbf{e}_2 = \mathbf{x}_2 - \mathbf{x}_3$:

$$G = J^T J = \begin{pmatrix} \mathbf{e}_1 & \mathbf{e}_2 \end{pmatrix}^T \begin{pmatrix} \mathbf{e}_1 & \mathbf{e}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{e}_1^T \\ \mathbf{e}_2^T \end{pmatrix} \begin{pmatrix} \mathbf{e}_1 & \mathbf{e}_2 \end{pmatrix} = \begin{pmatrix} \mathbf{e}_1 \cdot \mathbf{e}_1 & \mathbf{e}_1 \cdot \mathbf{e}_2 \\ \mathbf{e}_2 \cdot \mathbf{e}_1 & \mathbf{e}_2 \cdot \mathbf{e}_2 \end{pmatrix} = \begin{pmatrix} \|\mathbf{e}_1\|^2 & \mathbf{e}_1 \cdot \mathbf{e}_2 \\ \mathbf{e}_1 \cdot \mathbf{e}_2 & \|\mathbf{e}_2\|^2 \end{pmatrix}$$

whose invert is:

$$G^{-1} = \frac{1}{\|\mathbf{e}_1\|^2 \|\mathbf{e}_2\|^2 - (\mathbf{e}_1 \cdot \mathbf{e}_2)^2} \begin{pmatrix} \|\mathbf{e}_2\|^2 & -\mathbf{e}_1 \cdot \mathbf{e}_2 \\ -\mathbf{e}_1 \cdot \mathbf{e}_2 & \|\mathbf{e}_1\|^2 \end{pmatrix}$$

We also know that:

$$\|\mathbf{e}_1 \times \mathbf{e}_2\|^2 = \|\mathbf{e}_1\|^2 \|\mathbf{e}_2\|^2 - (\mathbf{e}_1 \cdot \mathbf{e}_2)^2$$

Then, we can write:

$$G^{-1} = \frac{1}{\|\mathbf{e}_1 \times \mathbf{e}_2\|^2} \begin{pmatrix} \|\mathbf{e}_2\|^2 & -\mathbf{e}_1 \cdot \mathbf{e}_2 \\ -\mathbf{e}_1 \cdot \mathbf{e}_2 & \|\mathbf{e}_1\|^2 \end{pmatrix}$$

It is now very easy to compute $\mathbf{u}$, and thus to compute $\nabla f = J\mathbf{u}$. The unit normal vector to K is $\mathbf{n} = \frac{\mathbf{e}_1 \times \mathbf{e}_2}{\|\mathbf{e}_1 \times \mathbf{e}_2\|}$.

5 Validation of the code

Most parts of the code will be tested using the eigenvectors of the Laplace operator on the sphere, via the Poisson solver. Since the Navier-Stokes solver is not easy to test, we will provide some physical insights by trying several values for the initial vorticity ω_0 and different viscosities ν . We will then aim to validate the results through global visualization.

5.1 Poisson solver

In order to test the code, we use the fact that harmonic polynomials in $\mathbb{R}^3$ are eigenvectors of the Laplace operator on the sphere. Specifically, a homogeneous harmonic polynomial p of degree l satisfies:

$$-\Delta_{S^2} p = l(l+1)p$$

We will test the code using the polynomials x , $x^2 - y^2$, and xz (corresponding to $l = 1$ and $l = 2$).

The L^2 norm of the error is $\|u_h - u_*\|_{L^2}$, where u_h is the solution computed with the current code, and u_* is the interpolation of the theoretical solution on the FEM basis. To find u_* , we only need to build the vector of the evaluations of the theoretical solution over all degrees of freedom, denoted U_* . This computation can be performed using the mass matrix M . We thus obtain the formula for the L^2 error:

$$\|u_h - u_*\|_{L^2} = \sqrt{(U_h - U_*)^T M (U_h - U_*)}$$

The H^1 error norm follows the exact same pattern and is given by the expression:

$$\|u_h - u_*\|_{H^1} = \sqrt{(U_h - U_*)^T M (U_h - U_*) + (U_h - U_*)^T S (U_h - U_*)}$$

It is worth mentioning that, for $\mathbb{P}_k$ FEM, we expect the convergence rates to be:

$$\|u_h - u_*\|_{L^2} \leq C_1 h^{k+1} |u_*|_{H^{k+1}}$$

and

$$\|u_h - u_*\|_{H^1} \leq C_2 h^k |u_*|_{H^{k+1}}$$

assuming that the solution has enough regularity. We report the relative errors in the following tables, which are computed as follows:

$$\delta = \frac{\|u_h - u_*\|}{\|u_*\|}$$

in both L^2 and H^1 norms, for both $\mathbb{P}_1$ and $\mathbb{P}_2$ finite elements.

	$h = 0.1$	$h = 0.05$	$h = 0.025$	$h = 0.01$	$h = 0.005$
x	2.1×10^{-3}	5.2×10^{-4}	1.3×10^{-4}	2.1×10^{-5}	5.2×10^{-6}
$x^2 - y^2$	5.3×10^{-3}	1.3×10^{-3}	3.3×10^{-4}	5.4×10^{-5}	1.3×10^{-5}
xz	4.7×10^{-3}	1.2×10^{-3}	2.3×10^{-4}	4.7×10^{-5}	1.2×10^{-5}

Table 3: L^2 -norm errors for $\mathbb{P}_1$

	$h = 0.1$	$h = 0.05$	$h = 0.025$	$h = 0.01$	$h = 0.005$
x	4.1×10^{-3}	1.0×10^{-3}	2.6×10^{-4}	4.4×10^{-5}	2.5×10^{-5}
$x^2 - y^2$	7.1×10^{-2}	3.7×10^{-3}	9.4×10^{-4}	1.5×10^{-4}	5.4×10^{-5}
xz	1.5×10^{-2}	3.8×10^{-3}	9.6×10^{-4}	1.6×10^{-4}	6.1×10^{-5}

Table 4: H^1 -norm errors for $\mathbb{P}_1$

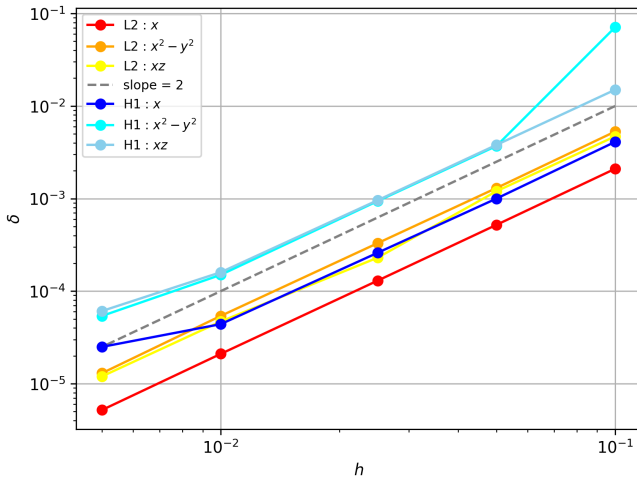
	$h = 0.1$	$h = 0.05$	$h = 0.025$	$h = 0.01$	$h = 0.005$
x	1.6×10^{-3}	3.9×10^{-4}	9.7×10^{-5}	1.6×10^{-5}	3.9×10^{-6}
$x^2 - y^2$	2.0×10^{-3}	5.1×10^{-4}	1.3×10^{-4}	2.1×10^{-5}	5.1×10^{-6}
xz	1.4×10^{-3}	3.6×10^{-4}	9.0×10^{-5}	1.5×10^{-5}	3.6×10^{-6}

Table 5: L^2 -norm errors for $\mathbb{P}_2$

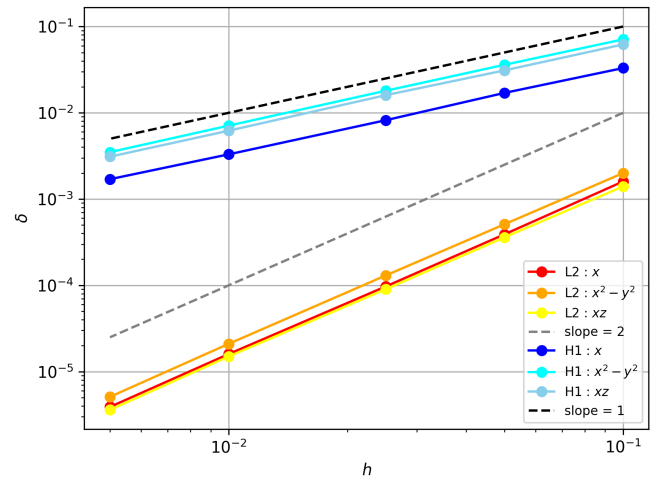
	$h = 0.1$	$h = 0.05$	$h = 0.025$	$h = 0.01$	$h = 0.005$
x	3.3×10^{-2}	1.7×10^{-2}	8.2×10^{-3}	3.3×10^{-3}	1.7×10^{-3}
$x^2 - y^2$	7.1×10^{-2}	3.6×10^{-2}	1.8×10^{-2}	7.1×10^{-3}	3.5×10^{-3}
xz	6.2×10^{-2}	3.1×10^{-2}	1.6×10^{-2}	6.2×10^{-3}	3.1×10^{-3}

Table 6: H^1 -norm errors

These results are summarized in the following two plots.



(a) L^2 and H^1 norms of the error for $\mathbb{P}_1$



(b) L^2 and H^1 norms of the error for $\mathbb{P}_2$

Figure 2: L^2 and H^1 norms of the error for $\mathbb{P}_1$ et $\mathbb{P}_2$

On subplot (a), we recognize the classical $\mathbb{P}_1$ convergence rate in the L^2 norm, which is $O(h^2)$. However, we also observe the same convergence rate in the H^1 norm, instead of the expected $O(h)$. This is a super-convergence result.

On subplot (b), we can see that we have lost one order of convergence compared to the theoretical expectations for $\mathbb{P}_2$. In fact, we observe an $O(h^2)$ rate instead of $O(h^3)$ for the L^2 norm, and an $O(h)$ rate instead of $O(h^2)$ for the H^1 norm. As mentioned in the project description, this likely stems from the approximation of the sphere by flat triangles.

5.2 Navier-Stokes solver

As mentioned at the beginning of this section, we will not test the code as explicitly as we did for the Poisson solver. Instead, we aim to validate the solver by interpreting the results through visualization tools.

We recall that we are solving the Navier-Stokes equations in their vorticity-stream function formulation:

$$\partial_t \omega + (\nabla^\perp \psi \cdot \nabla) \omega - \nu \Delta \omega = 0$$

where ψ satisfies the Poisson equation $\Delta \psi = \omega$, with an initial condition ω_0 prescribed by the user.

In what follows, we test the code on two different configurations: a viscous flow where diffusion dominates transport, and the inverse case. To do so, we must be able to compare the physical viscosity ν with other parameters. One approach is to write a non-dimensional version of the equation to highlight a dimensionless number—such as the Reynolds number—that can be used to compare these effects.

Let us consider a characteristic vorticity $\bar{\omega}$, stream function $\bar{\psi}$, length $\bar{L}$, and time $\bar{t}$. We define the dimensionless variables $\omega^* = \omega/\bar{\omega}$, $\psi^* = \psi/\bar{\psi}$, $\mathbf{x}^* = \mathbf{x}/\bar{L}$, and $t^* = t/\bar{t}$. Introducing the dimensionless operators ∇^* and Δ^* , the equation reads as:

$$\frac{\bar{\omega}}{\bar{t}} \partial_{t^*} \omega^* + \frac{\bar{\psi} \bar{\omega}}{\bar{L}^2} (\nabla^{*\perp} \psi^* \cdot \nabla^*) \omega^* - \frac{\nu \bar{\omega}}{\bar{L}^2} \Delta^* \omega^* = 0$$

By choosing the time scale $\bar{t} = \frac{\bar{L}^2}{\bar{\psi}}$, the equation can be rewritten as:

$$\partial_{t^*} \omega^* + (\nabla^{*\perp} \psi^* \cdot \nabla^*) \omega^* - \frac{\nu}{\bar{\psi}} \Delta^* \omega^* = 0$$

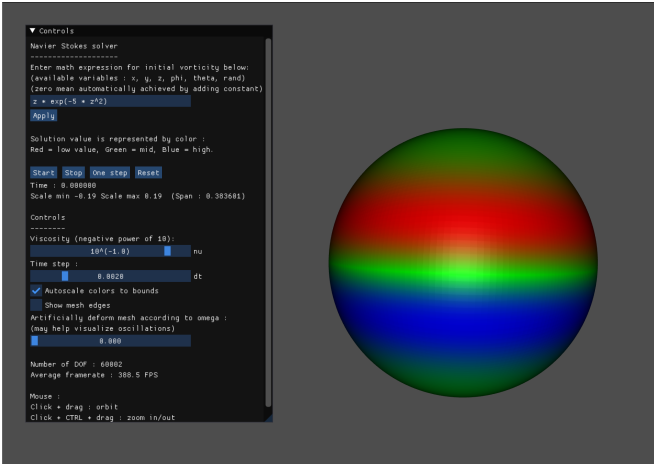
There are two regimes: if $\nu \gg \bar{\psi}$, the viscous term dominates the transport term; conversely, if $\nu \ll \bar{\psi}$, the transport term dominates the viscous term.

5.2.1 Viscous flow

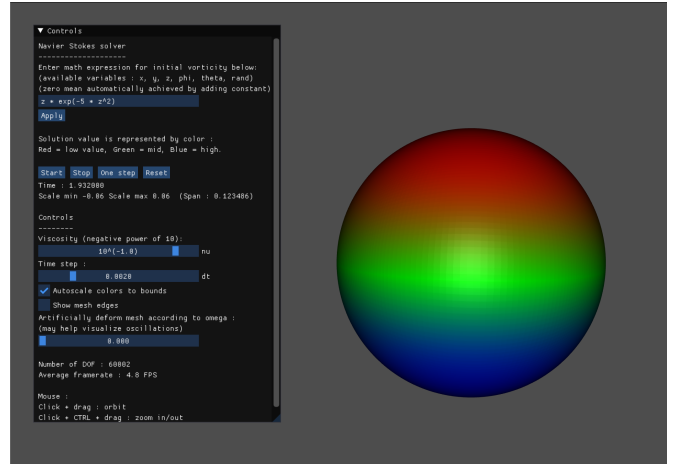
We begin with the case where the viscous term dominates the transport term. This simulation is easily performed by setting a large viscosity, as mentioned above. In order to obtain a clear visual interpretation, we can take the following initial condition:

$$\omega_0(x, y, z) = z e^{-z^2}$$

Typically, when running a simulation with $\nu \approx 10^{-1}$, we observe $\bar{\psi} \approx 10^{-12}$, such that $\nu \gg \bar{\psi}$. Under these conditions, we obtain the following results:



(a) $t = 0$



(b) $t = t_f$

Figure 3: Initial and final states of the Navier-Stokes problem with $\nu = 10^{-1}$ and $\omega_0(x, y, z) = ze^{-z^2}$

When the viscous effect dominates the transport term, the equation effectively reduces to a diffusion equation on the sphere:

$$\partial_t \omega = \nu \Delta \omega$$

which is equivalent to the heat equation. Given the initial condition, the vorticity diffuses across the sphere. The positive and negative lobes spread toward the north and south poles respectively, much like heat would redistribute. This behavior is clearly illustrated in Figure 3. Eventually, the solution decays to zero over time, as the system lacks a source term to sustain the energy. This mirrors a physical experiment where a sphere is heated at $t = 0$ and then left to cool; without a continuous heat source, its temperature eventually equilibrates with its surroundings (zero in our model).

5.2.2 Transport flow

Now, we set $\nu \ll \bar{\psi}$ to highlight the behavior of the transport term. This case is more challenging than the previous one because reducing ν requires a smaller time step to satisfy the CFL (Courant-Friedrichs-Lewy) condition. Consequently, the simulations are slower and can become less stable.

After testing several initial conditions and various values for ν , the results remain somewhat unsatisfying, as the simulations consistently develop significant numerical instabilities. However, it is worth noting that the integral of the transport term over the sphere is nearly zero (on the order of 10^{-10}). This indicates that the computed transport term does not introduce any artificial mass into the system. Furthermore, the $\mathbb{P}_2$ solver demonstrates superior stability compared to the $\mathbb{P}_1$ solver for advection-dominated flows; under identical configurations, the $\mathbb{P}_1$ solver quickly produces NaN values, whereas the $\mathbb{P}_2$ solver maintains a bounded solution where errors accumulate locally without immediate divergence.